



PROYECTO

2ºDesarrollo de Aplicaciones Multiplataforma
(DAM)

Marcos González, Cristian Tenorio y Diego Rodríguez

CONTENIDO

1. Introducción.....	2
2. Tecnologías utilizadas	2
3. Pasos de desarrollo	3
4. Mockups y diseño de pantallas.	4
5. Explicación de las actividades de la APP.....	5
SplashActivity.....	5
LoginActivity:	6
RegisterActivity:	11
ReviewDao.kt.....	15
AppDatabase.....	18
ReviewAdapter	20
MainActivity.....	24
ProfileActivity:	39
AndroidManifest.xml	41
6. Layouts (diseño)	43
7. Problemas encontrados y sus Soluciones	44
8. Diagrama de clases y arquitectura.	45
9. Video explicativo DE LA APP.....	¡Error! Marcador no definido.
10. Requisitos minimos de la app.....	47

1. INTRODUCCIÓN

El proyecto es una aplicación móvil Android llamada **MyList**, para gestionar reseñas de películas y series. Permite al usuario registrarse, iniciar sesión y añadir, editar o borrar reseñas, cada reseña incluye título, género y comentario, y se puede filtrar por género.

2. TECNOLOGÍAS UTILIZADAS

- Kotlin: lenguaje principal para Android.
- Android Studio: IDE utilizado.
- Room (SQLite): para almacenamiento local de reseñas.

```
implementation("androidx.room:room-runtime:2.8.1")
kapt("androidx.room:room-compiler:2.8.1")
implementation("androidx.room:room-ktx:2.8.1")
```

- Firebase Authentication: para gestión segura de usuarios.

```
implementation(platform( notation = "com.google.firebase:firebase-bom:34.3.0"))
implementation("com.google.firebase:firebase-storage")
implementation("com.google.firebase:firebase-auth")
implementation("com.google.firebase:firebase-firestore")
```

- Material Design 3: interfaz moderna y estilizada.
- RecyclerView: para mostrar la lista de reseñas dinámicamente.

```
implementation("androidx.appcompat:appcompat:1.7.0")
implementation("androidx.constraintlayout:constraintlayout:2.2.0")
implementation("androidx.recyclerview:recyclerview:1.3.1")
implementation("com.google.android.material:material:1.9.0")
```

- Glide: Cargar, mostrar y manejar imágenes desde URLs, recursos o archivos locales.

```
// Glide
implementation("com.github.bumptech.glide:glide:4.16.0")
kapt("com.github.bumptech.glide:compiler:4.16.0")
```

3. PASOS DE DESARROLLO

3.1 Configuración del proyecto

- Se creó un proyecto Android nuevo con Kotlin y tema Material3 DayNight NoActionBar en themes.xml.

- Configuramos Firebase:

- Descargamos el archivo google-services.json.
- Colocamos las dependencias en build.gradle (firebase-auth-ktx, firebase-firestore-ktx) y plugin com.google.gms.google-services.
- Esto permitió inicializar Firebase

3.2 Desarrollo de la autenticación

- LoginActivity: inicio de sesión del usuario.

- RegisterActivity: registro de usuario con validación básica.

- SplashActivity: detecta si el usuario tiene la sesión iniciada o no, es la actividad principal.

- Se utilizó FirebaseAuth para iniciar sesión o crear usuarios.

3.3 Implementación de la base de datos local

En ReviewDao se encuentra la creación de la tabla y los métodos:

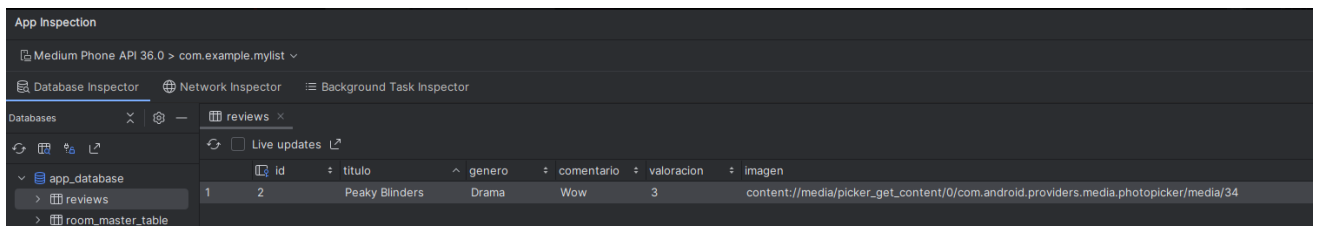
- Entidad Reviews: data class UserReview con id, título, género, comentario, valoración e imagen.

- DAO (UserReviewDao): funciones insert(), update(), delete(), getAll() y getByGenero().

Y luego la base de datos en:

- AppDatabase: extiende RoomDatabase y conecta al DAO.

- La base de datos se vería de esta manera:



3.4 Desarrollo de la pantalla principal

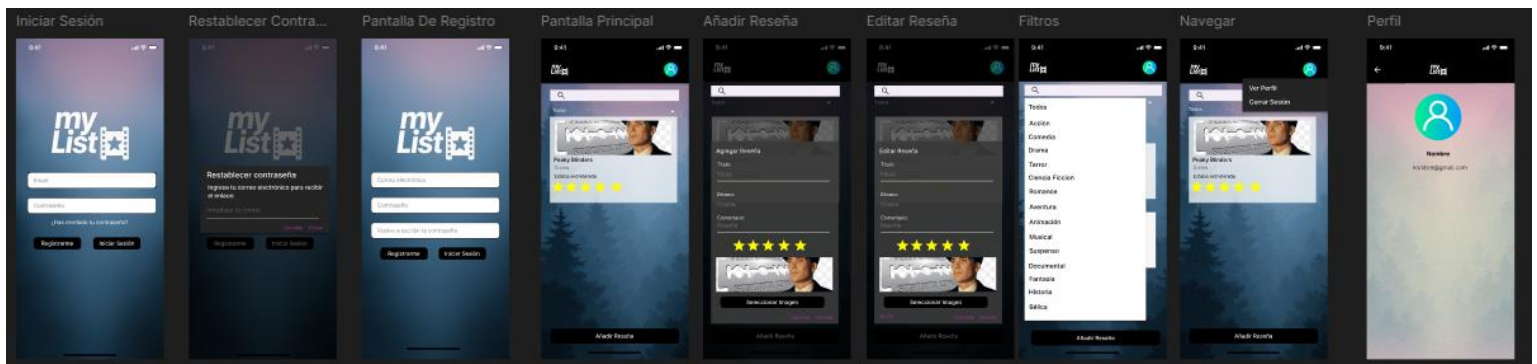
La interfaz principal es MainActivity que permite visualizar, añadir, editar, eliminar y filtrar reseñas por género, en esta actividad implementamos Room para realizar las operaciones CRUD a través de UserReviewDao, luego utilizamos un RecyclerView junto con el ReviewAdapter para mostrar la lista de reseñas con un layout personalizado item_resena.xml, y para el filtrado decidimos una barra de búsqueda para filtrar por nombre y usar un Spinner adaptado a un array que contiene los géneros para filtrar por género. Incluimos un botón para añadir una nueva reseña, el cual abre un diálogo para crearla, y al hacer clic sobre una reseña, se muestra otro diálogo que permite editarla o eliminarla. También hay un botón para cerrar sesión y una pequeña validación que, en caso de que el usuario no esté logueado, lo redirige a LoginActivity.

3.5 Desarrollo de la pantalla de perfil

Por último, queríamos hacer una pequeña página donde se mostrará una imagen del usuario, su nombre y su email.

4. MOCKUPS Y DISEÑO DE PANTALLAS.

El diseño de pantallas lo realizamos con **figma**



5. EXPLICACIÓN DE LAS ACTIVIDADES DE LA APP

SplashActivity

Qué hace: Es la pantalla inicial que decide a dónde enviar al usuario según si está logueado o no, no es visible.

Librerías:

- **import android.content.Intent:** Sirve para abrir otra pantalla o realizar una acción en el sistema, como enviar datos entre actividades.
- **import android.os.Bundle:** Permite guardar y recuperar información cuando una actividad se crea o se reinicia.
- **import androidx.appcompat.app.AppCompatActivity:** Es la clase base de las actividades que ofrece compatibilidad con versiones antiguas de Android.
- **import com.google.firebase.auth.FirebaseAuth:** Se usa para manejar el registro, inicio y cierre de sesión de usuarios en Firebase.

Flujo:

Cómo funciona (onCreate()):

- Revisa si **FirebaseAuth.currentUser** es null. (si el usuario no está logueado)
- Si no hay usuario: abre **LoginActivity**.
- Si ya hay usuario: abre **MainActivity**.
- Llama a **finish()** para cerrar el **SplashActivity**.

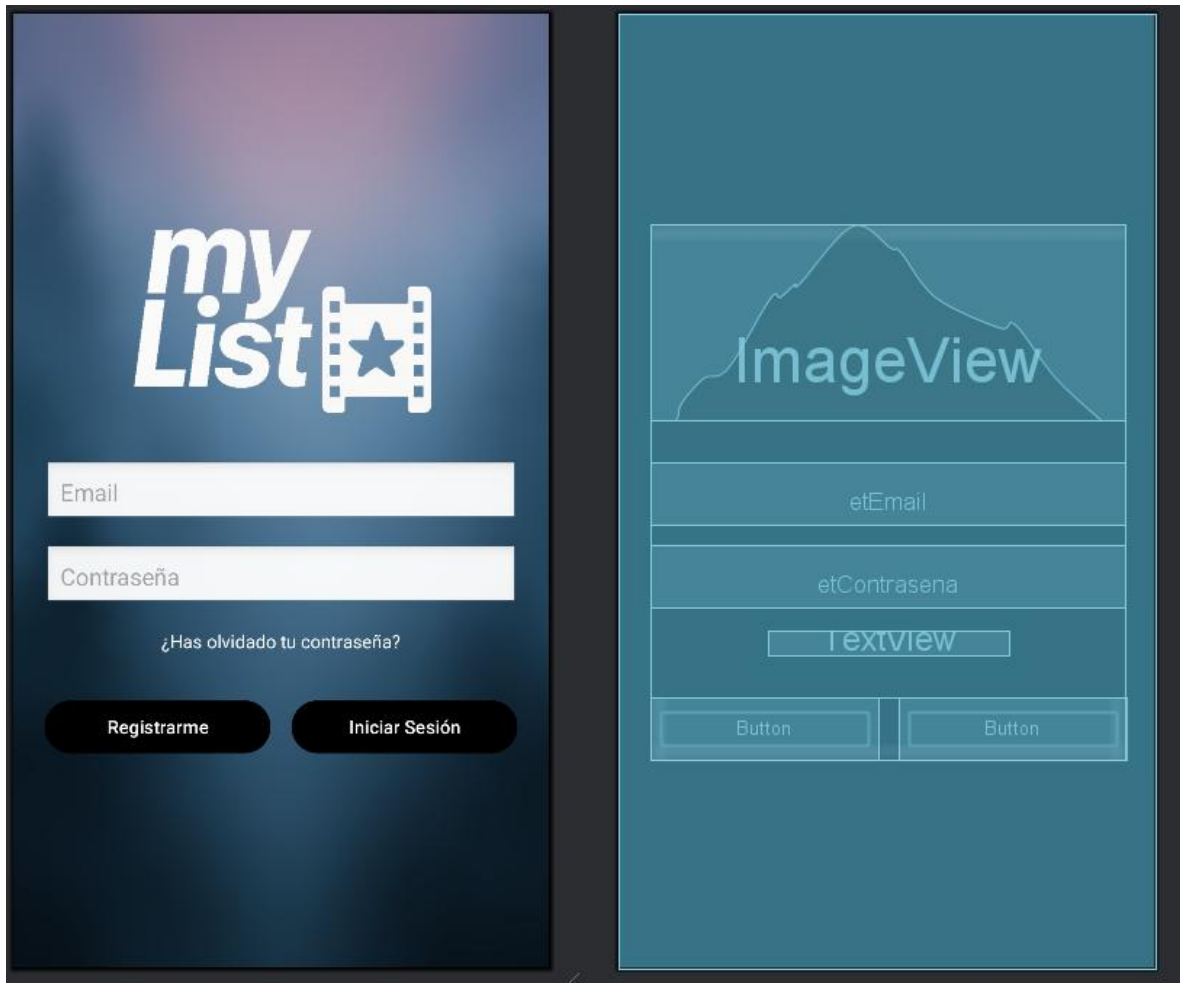
```
class SplashActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        val auth = FirebaseAuth.getInstance()  
        val currentUser = auth.currentUser  
  
        if (currentUser == null) {  
            // No está logueado  
            startActivity(Intent(packageContext = this, cls = LoginActivity::class.java))  
        } else {  
            // Ya logueado  
            startActivity(Intent(packageContext = this, cls = MainActivity::class.java))  
        }  
        finish()  
    }  
}
```

LoginActivity:

Qué hace: Permite a los usuarios iniciar sesión o acceder al apartado de registrarse.

Layout asociado (diseño): `activity_login.xml`

```
setContentView(R.layout.activity\_login)
```



Librerías:

- **import android.content.Intent:** Sirve para abrir otra pantalla o realizar una acción en el sistema, como enviar datos entre actividades.
- **import android.os.Bundle:** Permite guardar y recuperar información cuando una actividad se crea o se reinicia.
- **import android.widget.*:** Contiene los componentes visuales básicos de Android, como Button, TextView, EditText, etc.
- **import androidx.appcompat.app.AppCompatActivity:** Es la clase base de las actividades que ofrece compatibilidad con versiones antiguas de Android.
- **import com.google.firebase.FirebaseApp:** Inicializa Firebase en la aplicación para poder usar sus servicios.
- **import com.google.firebase.auth.FirebaseAuth:** Se usa para manejar el registro, inicio y cierre de sesión de usuarios en Firebase.

Componentes:

- **EditText etEmail y etContraseña:** ingresar email y contraseña.

```
2 usos
private lateinit var etEmail: EditText
2 usos
private lateinit var etContraseña: EditText
2 usos
```

- **Button btnLogin:** iniciar sesión.

```
private lateinit var btnLogin: Button
```

- **Button btnRegistrar:** abrir RegisterActivity.

```
private lateinit var btnRegistrar: Button
```

```
// Botón Registrar -> abrir la pantalla de registro
btnRegistrar.setOnClickListener {
    startActivity(Intent( packageContext = this, cls = RegisterActivity::class.java))
}
```

- **TextView tvOlvidar:** envía un correo para poder restablecer la contraseña

```
private lateinit var tvOlvidar: TextView
```

- **FirebaseAuth auth**: manejar la autenticación.

```
private lateinit var auth: FirebaseAuth
```

```
// Inicializar Firebase  
FirebaseApp.initializeApp(context = this)  
auth = FirebaseAuth.getInstance()
```

Para conectar las variables con los botones del layout.

```
etEmail = findViewById(id = R.id.etEmail)  
etContrasena = findViewById(id = R.id.etContrasena)  
btnLogin = findViewById(id = R.id.btnLogin)  
btnRegistrar = findViewById(id = R.id.btnRegistrar)  
tvOlvidar = findViewById(id = R.id.tvOlvidar)
```

Flujo:

- Botón Login: llama a **loginUsuario()**.

```
// Botón Login  
btnLogin.setOnClickListener { loginUsuario() }
```

- **TextView tvOlvidar:**

```
// Olvidaste contraseña
tvOlvidar.setOnClickListener {
    val emailInput = EditText(context = this).apply { hint = "Introduce tu correo" }

    AlertDialog.Builder(context = this)
        .setTitle("Restablecer contraseña")
        .setMessage("Ingresa tu correo electrónico para recibir el enlace:")
        .setView(emailInput)
        .setPositiveButton(text = "Enviar") { _, _ ->
            val email = emailInput.text.toString().trim()
            if (email.isEmpty()) {
                Toast.makeText(context = this, text = "Debes ingresar un correo", duration = Toast.LENGTH_SHORT).show()
                return@setPositiveButton
            }

            FirebaseAuth.getInstance().sendPasswordResetEmail(p0 = email)
                .addOnCompleteListener { task ->
                    val mensaje = if (task.isSuccessful)
                        "Correo de restablecimiento enviado a $email"
                    else
                        "Error: ${task.exception?.message}"
                    Toast.makeText(context = this, text = mensaje, duration = Toast.LENGTH_LONG).show()
                }
        }
        .setNegativeButton(text = "Cancelar", listener = null)
        .show()
}
```

Al hacer clic en el texto **¿Has olvidado tu contraseña?**, se abre un pequeño diálogo en el que los usuarios pueden restablecer su contraseña de **Firebase**.

En este diálogo se muestra un título: **“Restablecer contraseña”**, un mensaje con el texto **“Ingresa tu correo electrónico para recibir el enlace”**, y un campo de entrada con el hint **“Introduce tu correo”**, además de dos botones: **Enviar** y **Cancelar**.

El correo introducido no puede estar vacío y tiene que tener la estructura de prueba@prueba.com

Una vez enviado, se enviará automáticamente un correo con un enlace para cambiar la contraseña, y aparecerá un pequeño *toast* indicando que el correo ha sido enviado correctamente.

- El método **loginUsuario()** revisa que no haya campos vacíos
- Inicia sesión con Firebase (**auth.signInWithEmailAndPassword()**).
- Éxito: muestra mensaje y abre **MainActivity** (limpia las actividades para que no puedan darle al boton de volver, y aparecer en LoginActivity de nuevo).
- Error: muestra mensaje de error.

```
private fun loginUsuario() {
    val email = etEmail.text.toString().trim()
    val password = etContraseña.text.toString().trim()

    // Validación básica
    if (email.isEmpty() || password.isEmpty()) {
        Toast.makeText(context = this, text = "Por favor ingresa email y contraseña", duration = Toast.LENGTH_SHORT).show()
        return
    }

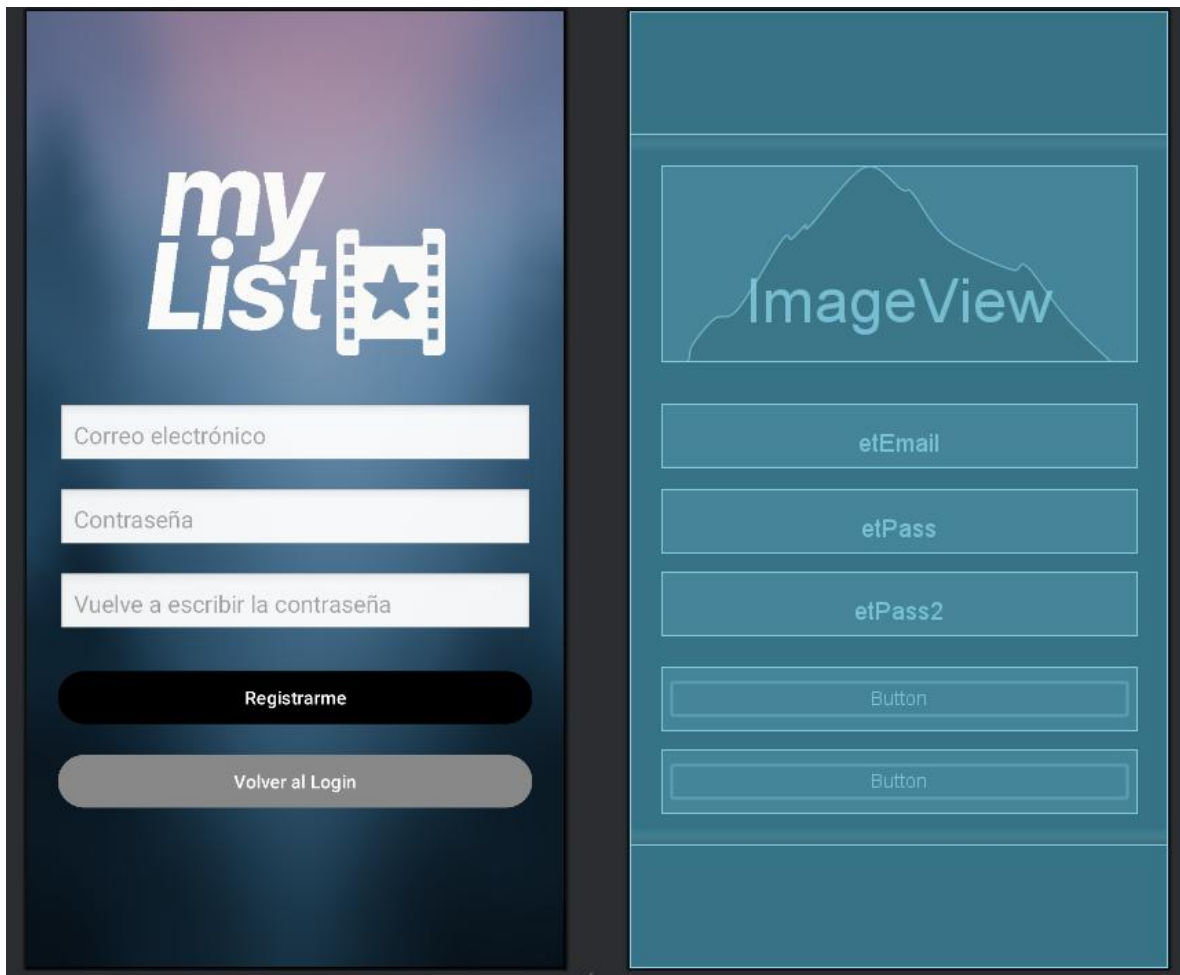
    // Iniciar sesión con Firebase
    auth.signInWithEmailAndPassword(p0 = email, p1 = password)
        .addOnCompleteListener { task ->
            if (task.isSuccessful) {
                Toast.makeText(context = this, text = "Login exitoso", duration = Toast.LENGTH_SHORT).show()
                // Navegar a MainActivity | si el inicio de sesión es exitoso (Tabulador) to complete
                val intent = Intent(packageContext = this, cls = MainActivity::class.java)
                intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
                startActivity(intent)
                finish()
            } else {
                // Mostrar error detallado
                Toast.makeText(
                    context = this,
                    text = "Error: ${task.exception?.localizedMessage}",
                    duration = Toast.LENGTH_LONG
                ).show()
            }
        }
    }
}
```

RegisterActivity:

Qué hace: Permite crear cuentas nuevas y guardar la información básica en Firestore o volver al Login.

Layout asociado (diseño): activity_register.xml

```
setContentView(R.layout.activity_register)
```



Librerías:

- **import android.content.Intent:** Sirve para abrir otra pantalla o realizar una acción en el sistema, como enviar datos entre actividades.
- **import android.os.Bundle:** Permite guardar y recuperar información cuando una actividad se crea o se reinicia.
- **import android.widget.Button:** Representa un botón que el usuario puede presionar para ejecutar una acción.
- **import android.widget.EditText:** Campo de texto que permite al usuario escribir información, como un nombre o una contraseña.
- **import android.widget.Toast:** Muestra mensajes cortos en pantalla para notificar al usuario (por ejemplo, "Inicio de sesión exitoso").
- **import androidx.appcompat.app.AppCompatActivity:** Es la clase base de las actividades que ofrece compatibilidad con versiones antiguas de Android.
- **import com.google.firebase.auth.FirebaseAuth:** Se usa para manejar el registro, inicio y cierre de sesión de usuarios en Firebase.
- **import com.google.firebase.firestore.FirebaseFirestore:** Permite conectarse y realizar operaciones (leer, escribir, actualizar, eliminar) en la base de datos Firestore de Firebase.

Componentes:

- **EditText etEmail, etPass, etPass2:** email y contraseñas.

```
private lateinit var etEmail: EditText
private lateinit var etPass: EditText
private lateinit var etPass2: EditText
```

- **Button btnRegistrar:** registrar usuario.

```
private lateinit var btnRegistrar: Button
```

- **Button btnVolverLogin:** volver a LoginActivity.

```
private lateinit var btnVolverLogin: Button
```

```
// Botón para volver al Login
btnVolverLogin.setOnClickListener {
    startActivity(Intent( packageContext = this, cls = LoginActivity::class.java))
    finish()
}
```

- **FirebaseAuth auth** : crear cuentas.

```
private lateinit var auth: FirebaseAuth
```

```
auth = FirebaseAuth.getInstance()
```

- **Firestore db** : guardar datos del usuario.

```
private val db: FirebaseFirestore by lazy { FirebaseFirestore.getInstance() }
```

Para conectar las variables con los botones del layout:

```
etEmail = findViewById( id = R.id.etEmail)
etPass = findViewById( id = R.id.etPass)
etPass2 = findViewById( id = R.id.etPass2)
btnRegistrar = findViewById( id = R.id.btnRegistrar)
btnVolverLogin = findViewById( id = R.id.btnVolverLogin)
```

Flujo:

- Botón Volver Login: abre **LoginActivity**.

```
// Botón para volver al Login
btnVolverLogin.setOnClickListener {
    startActivity(Intent( packageContext = this, cls = LoginActivity::class.java))
    finish()
}
```

- Botón Registrar: llama a **registrarUsuario()**.

```
// Botón para registrar usuario
btnRegistrar.setOnClickListener { registrarUsuario() }
}
```

- El método **registrarUsuario()** valida: campos vacíos, contraseñas iguales, longitud mínima.
- Crea usuario con **auth.createUserWithEmailAndPassword()**.
- Éxito: guarda email en Firestore y abre **MainActivity** (limpia las actividades para que no puedan darle al boton de volver, y aparecer en RegisterActivity de nuevo).
- Error: muestra mensaje de error.

```

private fun registrarUsuario() {
    val email = etEmail.text.toString().trim()
    val pass = etPass.text.toString().trim()
    val pass2 = etPass2.text.toString().trim()

    // Validaciones
    when {
        email.isEmpty() || pass.isEmpty() || pass2.isEmpty() -> {
            Toast.makeText(context = this, text = "Completa todos los campos", duration = Toast.LENGTH_SHORT).show()
            return
        }
        pass != pass2 -> {
            Toast.makeText(context = this, text = "Las contraseñas no coinciden", duration = Toast.LENGTH_SHORT).show()
            return
        }
        pass.length < 6 -> {
            Toast.makeText(context = this, text = "La contraseña debe tener al menos 6 caracteres", duration = Toast.LENGTH_SHORT).show()
            return
        }
    }
}

```

```

// Crear usuario en Firebase
auth.createUserWithEmailAndPassword(p0 = email, p1 = pass)
    .addOnCompleteListener { task ->
        if (task.isSuccessful) {
            auth.currentUser?.let { user ->
                // Guardar información opcional en Firestore
                val userData = hashMapOf(
                    "email" to email
                )
                db.collection(collectionPath = "users").document(documentPath = user.uid)
                    .set(userData)
            }
            Toast.makeText(context = this, text = "Usuario registrado", duration = Toast.LENGTH_SHORT).show()
            startActivity(Intent(packageContext = this, cls = MainActivity::class.java))
            finish()
        } else {
            Toast.makeText(
                context = this,
                text = "Error: ${task.exception?.message}",
                duration = Toast.LENGTH_LONG
            ).show()
        }
    }
}

```

ReviewDao.kt

Qué hace:

Define las operaciones principales de acceso a la base de datos local mediante Room, permite consultar, insertar, actualizar y eliminar reseñas de usuarios almacenadas en la tabla reviews, y se comunica con la clase AppDatabase y es utilizada desde MainActivity para gestionar la lista de reseñas mostrada al usuario.

Librerías:

- **import androidx.room.Dao:** Indica que la interfaz está marcada como Data Access Object (DAO), permite a Room generar automáticamente el código necesario para acceder a la base de datos y realizar las operaciones CRUD (crear, leer, actualizar y borrar).
- **import androidx.room.Delete:** Se utiliza para marcar un método dentro del DAO que eliminará un registro de la base de datos. Room genera automáticamente la instrucción SQL DELETE correspondiente.
- **import androidx.room.Entity:** Convierte una clase en una tabla de base de datos, cada propiedad de la clase se transforma en una columna dentro de esa tabla, y la clase UserReview es la entidad que representa la tabla reviews.
- **import androidx.room.Insert:** Marca un método del DAO para insertar datos en la tabla, Room genera el código SQL INSERT INTO de forma automática al compilar.
- **import androidx.room.PrimaryKey:** Define cuál campo de la entidad será la clave primaria de la tabla. Normalmente se usa con autoGenerate = true para que Room asigne automáticamente un valor único a cada fila.
- **import androidx.room.Query:** Permite escribir consultas SQL personalizadas dentro del DAO. Por ejemplo, @Query("SELECT * FROM reviews") devuelve todas las reseñas guardadas.
- **import androidx.room.Update:** Marca un método que actualizará registros existentes en la tabla según su clave primaria. Room genera el SQL UPDATE necesario de forma automática.

Componentes:

USERREVIEW.KT

- @Entity: convierte la clase en una tabla SQL llamada reviews, con los campos id, titulo, género, comentario, valoración e imagen, los cuales se vuelven columnas.
- @PrimaryKey (autoGenerate = true): id es la clave primaria y Room la autogenera con el valor por defecto 0L es típico para indicar que aún no tiene id antes de insertar

```
// Tabla reviews
36 usos
@Entity(tableName = "reviews") // Tabla
data class UserReview(
    @PrimaryKey(autoGenerate = true) val id: Long = 0L,
    val titulo: String,
    val genero: String,
    val comentario: String,
    val valoracion: Int,
    val imagen: String
)
```

USERREVIEWDAO.KT

- @Dao: Indica que la interfaz forma parte del sistema de acceso a datos de Room.

```
@Dao
interface UserReviewDao {
```

- @Query("SELECT * FROM reviews"): Devuelve todas las reseñas guardadas.

```
@Query( value = "SELECT * FROM reviews")
```

- @Query("SELECT * FROM reviews WHERE genero = :genero"): Devuelve reseñas filtradas por género.

```
@Query( value = "SELECT * FROM reviews WHERE genero = :genero")
```

- @Insert: Anotación del método de insert.

```
@Insert
```

- @Update: Anotación del método de update.

```
@Update
```

- @Delete: Anotación del método de delete.

```
⚡ @Delete
```

Funciones suspend:

Todas las funciones están marcadas como suspend para ejecutarse dentro de corrutinas (segundo plano) y no bloquear el hilo principal.

Para conectar con otras partes del proyecto:

- La interfaz UserReviewDao se instancia automáticamente a través de la clase AppDatabase.
- Desde MainActivity, se obtiene el DAO mediante db.userReviewDao() para ejecutar operaciones CRUD sobre las reseñas.
- Los métodos del DAO son llamados desde ViewModel o directamente en corrutinas dentro de la actividad principal.

Flujo:

- getAll(): Obtiene y muestra todas las reseñas guardadas en el RecyclerView.

```
suspend fun getAll(): List<UserReview>
```

- getByGenero(genero): Filtra la lista según el género seleccionado en el Spinner.

```
suspend fun getByGenero(genero: String): List<UserReview>
```

- insert(review): Se ejecuta al añadir una nueva reseña desde el diálogo "Añadir".

```
suspend fun insert(review: UserReview)
```

- update(review): Se ejecuta cuando el usuario edita una reseña existente.

```
suspend fun update(review: UserReview)
```

- delete(review): Se llama al eliminar una reseña mediante el botón correspondiente.

```
suspend fun delete(review: UserReview)
```

AppDatabase

Qué hace:

La clase AppDataBase implementa una base de datos local persistente usando ROOM, su diseño garantiza seguridad, eficiencia y escalabilidad, permitiendo que la aplicación gestione datos estructurados de manera fiable dentro del entorno.

```
@Database(entities = [UserReview::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    1 Usage
    abstract fun userReviewDao(): UserReviewDao

    4 Usages
    companion object {
        2 Usages
        @Volatile
        private var INSTANCE: AppDatabase? = null

        1 Usage
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(lock = this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    klass = AppDatabase::class.java,
                    name = "app_database"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Librerías:

- **android.content.Context:** proporciona acceso al entorno global de la aplicación, permitiendo obtener recursos, acceder a bases de datos o iniciar componentes del sistema.
- **androidx.room.Database:** define una clase como base de datos Room, indicando las entidades y la versión del esquema.
- **androidx.room.Room:** proporciona métodos para crear o acceder a la base de datos de manera sencilla y segura, gestionando automáticamente las conexiones.
- **androidx.room.RoomDatabase:** es la clase abstracta que actúa como punto principal de acceso a los datos, esta define la estructura de la base de datos y los métodos para obtener los DAO (Data Access Objects).

Componentes:

La clase se declara con **@Database**, que define las siguientes características:

- **entities =[UserReview:class]:** la base de datos tiene una tabla asociada a la entidad UserReview.
- **version = 1:** versión actual del esquema de base de datos.
- **exportSchema=false:** evita la exportación del esquema a un archivo externo (.json).

Flujo:

- **abstract fun userReviewDao():** UserReviewDao: este método proporciona acceso al objeto UserReviewDao, que contiene las operaciones CRUD (Create, Read, Update, Delete) sobre la entidad UserReview.
- **@Volatile** asegura que los cambios en la variable INSTANCE sean visibles a todos los hilos.
- El bloque **synchronized** garantiza la creación segura de la base de datos en entornos multihilos.
- **Room.databaseBuilder(...)** crea o recupera la base de datos denominada “app_database”
- Se utiliza **context.applicationContext** para evitar fugas de memoria, si la instancia ya existe, se reutiliza y en caso contrario, se crea una nueva.

Ventajas del uso de Room:

- Valida las consultas SQL en tiempo de compilación, y reduce el riesgo de errores y fugas de memoria.

ReviewAdapter

Qué hace:

El archivo ReviewAdapter define el adaptador encargado de mostrar la lista de reseñas guardadas en la base de datos dentro de un **RecyclerView**, su función principal es vincular los datos del modelo UserReview con el diseño visual de cada ítem (item_resena.xml), permitiendo manejar acciones cuando el usuario selecciona una reseña.

Librerías:

- **Import UserReview:** Importa la clase UserReview que representa la entidad o modelo de datos donde se guarda la información de cada reseña (título, género y comentario), esto permite usarla dentro del adaptador.
- **import android.view.LayoutInflater:** Sirve para convertir (inflar) un archivo XML de diseño en un objeto View que se puede mostrar en pantalla, es usado en RecyclerView para crear cada elemento de la lista.
- **import android.view.View:** Es la clase base de todos los componentes visuales de Android (botones, textos, etc.), esta permite manejar acciones como clics, visibilidad o estilos de cualquier elemento gráfico.
- **Import android.view.ViewGroup:** Representa un contenedor que puede tener dentro otras vistas (View), en el caso del RecyclerView se usa para colocar correctamente cada elemento dentro de la lista.
- **import android.widget.TextView:** Permite mostrar texto en la interfaz de usuario en un RecyclerView, y se usa para enseñar el título, género o comentario de una reseña.
- **import androidx.recyclerview.widget.RecyclerView:** Es el componente de Android que muestra listas o colecciones de elementos, y se usa junto con un adapter y un ViewHolder para mostrar los datos de UserReview en forma de lista o tarjetas.

Componentes:

- List<UserReview> reviews : Lista de reseñas cargadas desde la base de datos.

```
private var reviews: List<UserReview>,
```

- onClick: (UserReview) -> Unit Función lambda que define qué ocurre al pulsar sobre una reseña.

```
private val onClick: (UserReview) -> Unit
```

- RecyclerView.Adapter<ReviewAdapter.ReviewViewHolder>: Es la clase base que conecta la lista de reseñas con la interfaz.

```
) : RecyclerView.Adapter<ReviewAdapter.ReviewViewHolder>() {
```

- ReviewViewHolder: Es la clase interna que representa cada ítem de la lista (vista individual).

```
class ReviewViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
```

- TextView titulo : Muestra el título de la reseña.

```
val titulo: TextView = itemView.findViewById( id = R.id.textTitulo)
```

- TextView genero: Muestra el género asociado a la reseña.

```
val genero: TextView = itemView.findViewById( id = R.id.textGenero)
```

- TextView comentario : Muestra el comentario del usuario.

```
val comentario: TextView = itemView.findViewById( id = R.id.textComentario)
```

- ratingBar: Muestra las estrellas de las valoraciones de las reseñas.

```
val ratingBar: RatingBar = itemView.findViewById( id = R.id.ratingBarItem)
```

- imageView: Muestra la imagen guardada en la galería.

```
val imageView: ImageView = itemView.findViewById( id = R.id.imageResena)
```

Para conectar las variables con los elementos del layout:

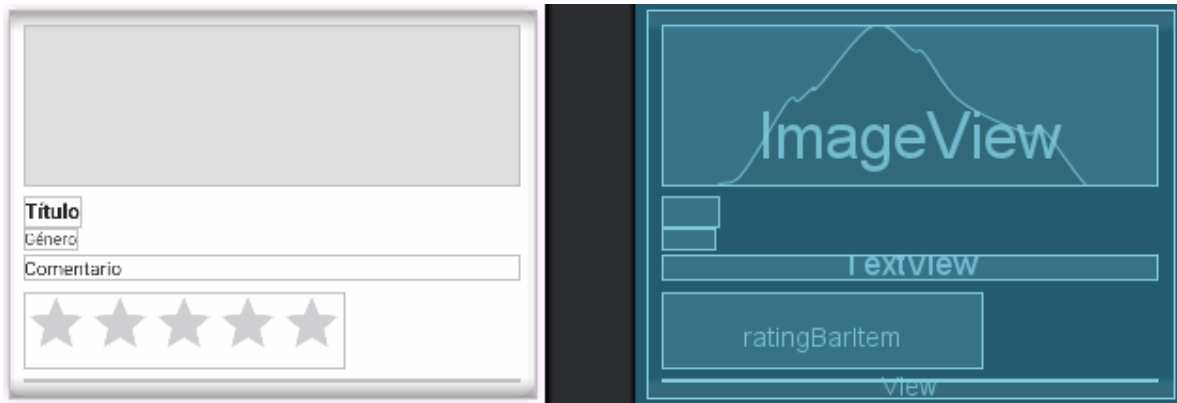
En el método onCreateViewHolder, hace que el layout se infle (item_resena.xml) con LayoutInflater y se asocian los TextView con cada campo (textTitulo, textGenero, textComentario). Cada instancia de ReviewViewHolder contiene las referencias a estos elementos para poder asignarles los valores del modelo UserReview.

Flujo:

- onCreateViewHolder()
 - Se ejecuta cuando el RecyclerView necesita crear una nueva vista.
 - Infla el diseño item_resena.xml y devuelve un ReviewViewHolder asociado.

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ReviewViewHolder {
    val view = LayoutInflater.from( context = parent.context)
        .inflate( resource = R.layout.item_resena, root = parent, attachToRoot = false)
    return ReviewViewHolder( itemView = view)
}
```

Layout del diseño de la reseña (**item_resena.xml**) :



- **onBindViewHolder()**

Asigna los valores del objeto UserReview a los TextView del ítem, y configura el evento setOnClickListener para detectar clics sobre cada reseña y ejecutar la acción pasada como parámetro.

El código carga la imagen de review.imagen en el ImageView con Glide y asigna un clic al elemento para ejecutar onClick(review).

```
override fun onBindViewHolder(holder: ReviewViewHolder, position: Int) {  
    val review = reviews[position]  
    holder.titulo.text = review.titulo  
    holder.genero.text = review.genero  
    holder.comentario.text = review.comentario  
    holder.ratingBar.rating = review.valoracion.toFloat()  
  
    Glide.with(context = holder.itemView.context)  
        .load(string = review.imagen)  
        .centerCrop()  
        .placeholder(resourceId = R.drawable.placeholder)  
        .into(holder.imageView)  
  
    holder.itemView.setOnClickListener { onClick(review) }  
}
```

- **getItemCount()**

Devuelve el número total de reseñas disponibles en la lista.

```
override fun getItemCount(): Int = reviews.size
```

- **actualizarDatos(newList: List<UserReview>)**

Permite actualizar la lista de reseñas (reviews) y refrescar el RecyclerView con los nuevos datos mediante notifyDataSetChanged().

```
fun actualizarDatos(newList: List<UserReview>) {  
    reviews = newList  
    notifyDataSetChanged()  
}
```

MainActivity

Qué hace:

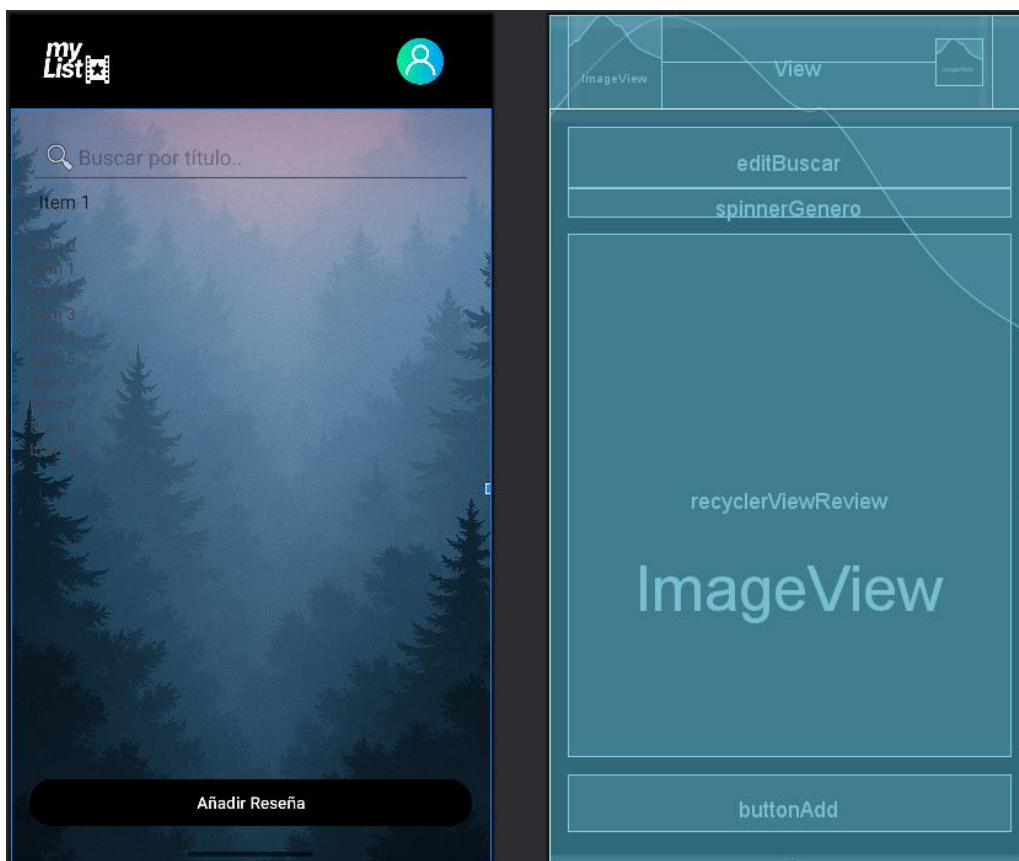
La clase MainActivity es la actividad principal, cuya función es gestionar reseñas de películas/series.

El código gestiona el ciclo de vida principal de la aplicación, y permite al usuario una vez autenticado realizar las siguientes acciones:

- Añadir, editar y eliminar reseñas.
- Filtrar las reseñas por género o nombre.
- Ver los datos almacenados en una base de datos local.
- Ver perfil o Cerrar sesión.

Layout asociado (diseño): activity_main.xml

```
setContentView(R.layout.activity_main)
```



Librerías:

- **import AppDatabase:** Permite acceder a la base de datos local de la app.
- **import UserReview y import UserReviewDao:** Definen la entidad de reseña y las operaciones CRUD en la base de datos.
- **import android.Manifest:** Contiene constantes para manejar permisos del sistema Android.
- **import android.app.NotificationChannel y import NotificationManager:** Permiten crear y gestionar canales de notificación en Android.
- **import android.content.Context:** Proporciona contexto para acceder a recursos y servicios del sistema.
- **import android.content.Intent:** Sirve para abrir otras actividades o iniciar servicios.
- **import android.content.pm.PackageManager:** Permite verificar permisos y características del dispositivo.
- **import android.net.Uri:** Representa direcciones URI para archivos o recursos web.
- **import android.os.Build:** Permite verificar la versión de Android en tiempo de ejecución.
- **import android.os.Bundle:** Guarda y recupera el estado de la actividad.
- **import android.widget.*:** Contiene componentes de interfaz como Button, Spinner, EditText, ImageView, RatingBar.
- **import androidx.activity.result.contract.ActivityResultContracts:** Facilita manejar resultados de actividades, como la autorización de permisos.
- **import androidx.annotation.RequiresPermission:** Anota métodos que requieren permisos específicos para funcionar.
- **import androidx.appcompat.app.AlertDialog:** Permite mostrar cuadros de diálogo para añadir o editar reseñas.
- **import androidx.appcompat.app.AppCompatActivity:** Clase base de la actividad con compatibilidad hacia atrás.
- **import androidx.core.app.ActivityCompat:** Proporciona métodos para manejar permisos en distintas versiones de Android.
- **Import androidx.core.app.NotificationCompat y import NotificationManagerCompat:** Permiten crear y mostrar notificaciones de forma compatible en distintas versiones de Android.
- **import androidx.lifecycle.lifecycleScope:** Permite ejecutar operaciones de manera asíncrona usando corrutinas vinculadas al ciclo de vida de la actividad.
- **Import androidx.recyclerview.widget.LinearLayoutManager y import RecyclerView:** Permiten mostrar listas desplazables de reseñas.
- **import com.bumptech.glide.Glide:** Carga y muestra imágenes de manera eficiente.
- **import com.google.firebase.FirebaseApp:** Inicializa Firebase en la aplicación.
- **import com.google.firebase.auth.FirebaseAuth:** Maneja la autenticación de usuarios con Firebase.
- **import kotlinx.coroutines.Dispatchers, launch y withContext:** Permiten ejecutar tareas en segundo plano y actualizar la UI de forma segura.
- **import androidx.core.widget.addTextChangedListener:** Sirve para detectar cambios en un EditText usando una función corta.

Componentes:

- **Autenticación Firebase:**

Se inicializa Firebase y se obtiene una instancia de FirebaseAuth.

```
// Inicializar Firebase
FirebaseApp.initializeApp(context = this)
auth = FirebaseAuth.getInstance()
```

Se comprueba si existe un usuario autenticado, si no es así se redirige a la pantalla de inicio de sesión (LoginActivity):

```
// Verificar si hay usuario logueado
val currentUser = auth.currentUser
if (currentUser == null) {
    val intent = Intent(packageContext = this, cls = LoginActivity::class.java)
    intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
    startActivity(intent)
    finish()
    return
}
```

- **Base de datos local con Room:**

```
// Configurar base de datos
db = AppDatabase.getDatabase(context = this)
reviewDao = db.userReviewDao()
```

Se obtiene una instancia de la base de datos AppDatabase, y se accede al UserReviewDao, que es el encargado de ejecutar las operaciones CRUD sobre la entidad UserReview.

- Interfaz de usuario:

```
addButton = findViewById( id = R.id.buttonAdd)
generoFiltro = findViewById( id = R.id.spinnerGenero)
recyclerView = findViewById( id = R.id.recyclerViewReview)
```

Los elementos de la interfaz se vinculan con los elementos del diseño activity_main.xml.

- RecyclerView y Adaptador

```
// Configurar RecyclerView
recyclerView.layoutManager = LinearLayoutManager( context = this)
adapter = ReviewAdapter(reviews) { review -> mostrarDialogoEditar( resena = review) }
recyclerView.adapter = adapter
```

El recyclerview muestra la lista de reseñas almacenadas, usando un adaptador personalizado (reviewadapter) que actualiza los datos dinámicamente y permite editar reseñas al seleccionarlás.

- Barra de Búsqueda (Filtro por nombre):

```
// Carga todas las reseñas al iniciar (método)
cargarTodas()

// Lógica del Buscador
editBuscar.addTextChangedListener { texto ->
    val query = texto.toString()
    if (query.isBlank()) {
        cargarTodas()
    } else {
        buscarPorTitulo( nombre = query)
    }
}
```

- Carga todas las reseñas al principio, si no hay nada escrito en la barra de búsqueda, carga todo, si hay algo escrito busca por el nombre del título de la reseña.
- La instrucción verifica si el dispositivo usa Android 13 o superior y, si la app no tiene permiso para enviar notificaciones, solicita dicho permiso al usuario.

```
//Pedir permiso para recibir notificaciones

if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    if (checkSelfPermission(permission = android.Manifest.permission.POST_NOTIFICATIONS)
        != PackageManager.PERMISSION_GRANTED
    ) {
        requestPermissions(permissions = arrayOf(android.Manifest.permission.POST_NOTIFICATIONS), requestCode = 101)
    }
}

// Crear Notificaciones
crearCanalNotificacion()
```

- El método **crearCanalNotificacion ()**, crea un canal de notificaciones llamado "Reseñas" para dispositivos con Android 8 o superior, que es necesario para poder mostrar las notificaciones.
- El método **mostrarNotificacion (titulo, mensaje)**, construye y muestra una notificación usando ese canal con el título y mensaje proporcionados, siempre que el usuario de los permisos necesarios a la app para enviar notificaciones.

```
1 uso
private fun crearCanalNotificacion() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val nombre = "Reseñas"
        val descripcion = "Notificaciones"
        val importancia = NotificationManager.IMPORTANCE_DEFAULT
        val canal = NotificationChannel(id = "RESEÑAS", name = nombre, importance = importancia).apply {
            description = descripcion
        }

        val notificationManager: NotificationManager =
            getSystemService(name = Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.createNotificationChannel(channel = canal)
    }
}

2 usos
@RequiresPermission(value = Manifest.permission.POST_NOTIFICATIONS)
private fun mostrarNotificacion(titulo: String, mensaje: String) {
    val builder = NotificationCompat.Builder(context = this, channelId = "RESEÑAS")
        .setSmallIcon(R.drawable.notification)
        .setContentTitle(titulo)
        .setContentText(mensaje)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setAutoCancel(true)

    val notificationManager = NotificationManagerCompat.from(context = this)
    notificationManager.notify(id = System.currentTimeMillis().toInt(), notification = builder.build())
}
```

- **Spinner (Filtro de género):**

```
val generosFiltro = resources.getStringArray( id = R.array.generos_array)

generoFiltro.adapter =
    ArrayAdapter( context = this, resource = android.R.layout.simple_spinner_dropdown_item, objects = generosFiltro)
```

El spinner permite filtrar las reseñas por género (acción, comedia, drama, etc), al seleccionar un género se ejecuta el método filtrarLista()

Los géneros del spinner se obtienen de un array que se encuentra en res/values/strings.xml.

```
<string-array name="generos_array">
    <item>Todos</item>
    <item>Accion</item>
    <item>Comedia</item>
    <item>Drama</item>
    <item>Terror</item>
    <item>Ciencia Ficción</item>
    <item>Romance</item>
    <item>Aventura</item>
    <item>Animación</item>
    <item>Musical</item>
    <item>Suspenso</item>
    <item>Documental</item>
    <item>Fantasia</item>
    <item>Historia</item>
    <item>Bélica</item>
</string-array>
```

Flujo:

- **imageProfile:** Icono de perfil en la pantalla principal, al hacer click, muestra un PopupMenu con dos opciones:
 - Ver perfil: Abre ProfileActivity para mostrar la información del usuario.
 - Cerrar sesión: Cierra la sesión del usuario con auth.signOut(), redirige a LoginActivity y cierra MainActivity.

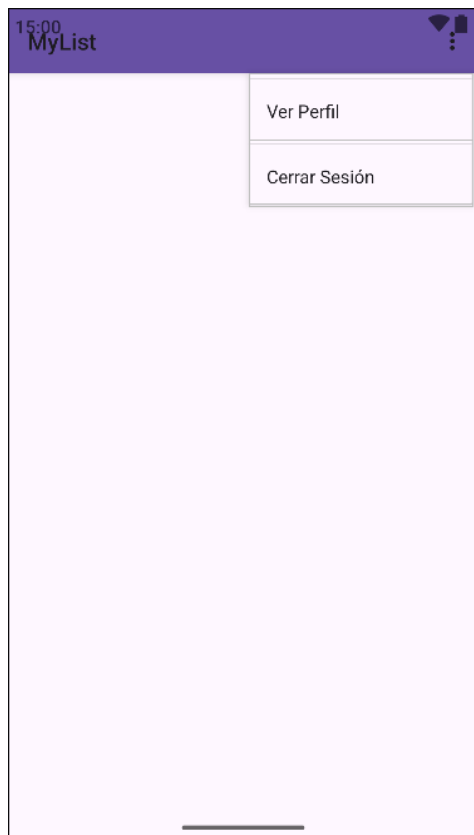
```

val imageProfile = findViewById<ImageView>(id = R.id.imageProfile)
imageProfile.setOnClickListener { view ->
    PopupMenu(context = this, anchor = view).apply {
        menuInflater.inflate(menuRes = R.menu.profile_menu, menu)

        setOnMenuItemClickListener { item ->
            when (item.itemId) {
                R.id.menu_ver_perfil -> {
                    startActivity(Intent(packageContext = this@MainActivity, cls = ProfileActivity::class.java))
                    true
                }
                R.id.menu_cerrar_sesion -> {
                    auth.signOut()
                    Intent(packageContext = this@MainActivity, cls = LoginActivity::class.java).also { intent ->
                        intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
                    }.startActivity(intent)
                    finish()
                    true
                }
            }
            else -> false
        }
    }
    show()
}
}

```

El menuInflater que es el popupmenu se obtiene de res/menú/profile_menu.xml.



- Filtrado de reseñas por nombre:

```
// Buscar por título (Buscador)
1 uso
private fun buscarPorTitulo(nombre: String) {
    lifecycleScope.launch {
        val resultados = reviewDao.getByTitulo(nombre)
        adapter.actualizarDatos( newList = resultados)
    }
}

// Cargar todas las reseñas (Buscador)
2 usos
private fun cargarTodas() {
    lifecycleScope.launch {
        val lista = reviewDao.getAll()
        adapter.actualizarDatos( newList = lista)
    }
}
}
```

Esta lógica la utiliza la barra de búsqueda, estos dos métodos, el primero cargarTodas(), que muestra toda la lista de reseñas, y el segundo, buscarPorTitulo que filtra las reseñas por el nombre del título.

- Filtrado de reseñas por género:

```
// Lógica del Spinner
5 usos
private fun filtrarLista(genero: String) {
    lifecycleScope.launch( context = Dispatchers.IO) {
        val lista = if (genero == "Todos") reviewDao.getAll() else reviewDao.getByGenero(genero)
        withContext( context = Dispatchers.Main) {
            reviews = lista
            adapter.actualizarDatos( newList = reviews)
        }
    }
}
```

Utiliza lifecycleScope y Dispatchers.IO para acceder a la base de datos sin bloquear el hilo principal, y los resultados se muestran en la interfaz a través del adaptador.

- Añadir una nueva reseña:

```
private fun mostrarDialogoAñadir() {
    val dialogView = LayoutInflater.inflate( resource = R.layout.dialog_resena, root = null)
    val tituloInput = dialogView.findViewById<EditText>( id = R.id.editTitulo)
    val comentarioInput = dialogView.findViewById<EditText>( id = R.id.editComentario)
    val generoSpinner = dialogView.findViewById<Spinner>( id = R.id.spinnerGenero)
    val ratingBar = dialogView.findViewById<RatingBar>( id = R.id.ratingBarValoracion)
    val imageView = dialogView.findViewById<ImageView>( id = R.id.imagePreview)
    val btnSeleccionarImagen = dialogView.findViewById<Button>( id = R.id.btnSeleccionarImagen)

    vistaPreviaTemporal = imageView
    imagenSeleccionada = null

    val generos = resources.getStringArray( id = R.array.generos_array).drop( n = 1)
    val adapter = ArrayAdapter( context = this, resource = android.R.layout.simple_spinner_item, objects = generos)
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
    generoSpinner.adapter = adapter

    btnSeleccionarImagen.setOnClickListener {
        selectorImagenLauncher.launch( input = "image/*")
    }

    val dialog = AlertDialog.Builder( context = this)
        .setTitle("Añadir Reseña")
        .setView(dialogView)
        .setPositiveButton( text = "Guardar", listener = null)
        .setNegativeButton( text = "Cancelar", listener = null)
        .create()
}
```

```

dialog.setOnShowListener {
    val botonGuardar = dialog.getButton( whichButton = AlertDialog.BUTTON_POSITIVE)
    botonGuardar.setOnClickListener {
        val titulo = tituloInput.text.toString().trim()
        val comentario = comentarioInput.text.toString().trim()
        val genero = generoSpinner.selectedItem?.toString()?.trim() ?: ""
        val valoracion = ratingBar.rating
        val imagen = imagenSeleccionada

        when {
            titulo.isEmpty() -> Toast.makeText(
                context = this,
                text = "El título no puede estar vacío",
                duration = Toast.LENGTH_SHORT
            ).show()

            comentario.isEmpty() -> Toast.makeText(
                context = this,
                text = "El comentario no puede estar vacío",
                duration = Toast.LENGTH_SHORT
            ).show()

            valoracion == 0f -> Toast.makeText(
                context = this,
                text = "Debes dar una valoración",
                duration = Toast.LENGTH_SHORT
            ).show()

            imagen.isNullOrEmpty() -> Toast.makeText(
                context = this,
                text = "Debes seleccionar una imagen",
                duration = Toast.LENGTH_SHORT
            ).show()

            else -> {
                val resena = UserReview(
                    titulo = titulo,

```

```

        genero = genero,
        comentario = comentario,
        valoracion = valoracion.toInt(),
        imagen = imagen
    )

    lifecycleScope.launch( context = Dispatchers.IO) {
        reviewDao.insert( review = resena)
        filtrarLista(generoFiltro.selectedItem.toString())

        // Mostrar notificación en el hilo principal
        withContext( context = Dispatchers.Main) {
            if (ActivityCompat.checkSelfPermission(
                context = this@MainActivity,
                permission = Manifest.permission.POST_NOTIFICATIONS
            ) != PackageManager.PERMISSION_GRANTED
            ) {
                // Si no hay permiso, lo pedimos
                ActivityCompat.requestPermissions(
                    this@MainActivity,
                    permissions = arrayOf(Manifest.permission.POST_NOTIFICATIONS),
                    requestCode = 101
                )
                return@withContext
            }

            mostrarNotificacion( titulo = "Nueva reseña añadida", mensaje = "Has añadido: $titulo")
        }
    }

    dialog.dismiss()
}
}
}
}
}

```

- Muestra un cuadro de diálogo con campos para título, género, comentario y valoración, al confirmar, se crea un objeto UserReview y se inserta en la base de datos mediante `reviewDao.insert(resena)`, y tiene una pequeña validación en los campos, no pueden estar vacíos, y una vez agregada la reseña, el hilo principal, verifica si la app tiene permiso para enviar notificaciones, si no lo tiene, lo solicita al usuario, y si sí lo tiene, muestra una notificación con el título “Nueva reseña añadida” y un mensaje diciendo “Has añadido: x película/serie”.

- Editar o eliminar una reseña existente:

```
private fun mostrarDialogoEditar(resena: UserReview) {
    val dialogView = inflater.inflate( resource = R.layout.dialog_resena, root = null)
    val tituloInput = dialogView.findViewById<EditText>( id = R.id.editTitulo)
    val comentarioInput = dialogView.findViewById<EditText>( id = R.id.editComentario)
    val generoSpinner = dialogView.findViewById<Spinner>( id = R.id.spinnerGenero)
    val ratingBar = dialogView.findViewById<RatingBar>( id = R.id.ratingBarValoracion)
    val imageView = dialogView.findViewById<ImageView>( id = R.id.imagePreview)
    val btnSeleccionarImagen = dialogView.findViewById<Button>( id = R.id.btnSeleccionarImagen)

    vistaPreviaTemporal = imageView
    imagenSeleccionada = resena.imagen

    val generos = resources.getStringArray( id = R.array.generos_array).drop( n = 1)
    val adapter = ArrayAdapter( context = this, resource = android.R.layout.simple_spinner_item, objects = generos)
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
    generoSpinner.adapter = adapter

    tituloInput.setText(resena.titulo)
    comentarioInput.setText(resena.comentario)
    ratingBar.rating = resena.valoracion.toFloat()

    if (resena.imagen.isNotEmpty()) {
        Glide.with( activity = this)
            .load(Uri.parse( urlString = resena.imagen))
            .centerCrop()
            .placeholder( resourceId = R.drawable.placeholder)
            .into(imageView)
    }

    val pos = generos.indexOf(resena.genero)
    if (pos >= 0) generoSpinner.setSelection(pos)

    btnSeleccionarImagen.setOnClickListener {
        selectorImagenLauncher.launch( input = "image/*")
    }
}
```

```

val dialog = AlertDialog.Builder( context = this)
    .setTitle("Editar Reseña")
    .setView(dialogView)
    .setPositiveButton( text = "Guardar", listener = null)
    .setNegativeButton( text = "Borrar", listener = null)
    .setNeutralButton( text = "Cancelar", listener = null)
    .create()

dialog.setOnShowListener {

    val botonGuardar = dialog.getButton( whichButton = AlertDialog.BUTTON_POSITIVE)
    val botonBorrar = dialog.getButton( whichButton = AlertDialog.BUTTON_NEUTRAL)

    botonGuardar.setOnClickListener {

        val titulo = tituloInput.text.toString().trim()
        val comentario = comentarioInput.text.toString().trim()
        val genero = generoSpinner.selectedItem?.toString()?.trim() ?: resena.genero
        val valoracion = ratingBar.rating
        val imagen = imagenSeleccionada

        when {

            titulo.isEmpty() -> Toast.makeText(
                context = this,
                text = "El titulo no puede estar vacio",
                duration = Toast.LENGTH_SHORT
            ).show()

            comentario.isEmpty() -> Toast.makeText(
                context = this,
                text = "El comentario no puede estar vacio",
                duration = Toast.LENGTH_SHORT
            ).show()

            valoracion == 0f -> Toast.makeText(
                context = this,
                text = "Debes dar una valoración",
                duration = Toast.LENGTH_SHORT
            ).show()

            imagen.isNullOrEmpty() -> Toast.makeText(

```

```

        context = this,
        text = "Debes seleccionar una imagen",
        duration = Toast.LENGTH_SHORT
    ).show()

    else -> {
        val resenaActualizada = resena.copy(
            titulo = titulo,
            genero = genero,
            comentario = comentario,
            valoracion = valoracion.toInt(),
            imagen = imagen
        )

        lifecycleScope.launch( context = Dispatchers.IO) {
            reviewDao.update( review = resenaActualizada)
            filtrarLista(generoFiltro.selectedItem.toString())

            // Mostrar notificación en el hilo principal
            withContext( context = Dispatchers.Main) {
                if (ActivityCompat.checkSelfPermission(
                    context = this@MainActivity,
                    permission = Manifest.permission.POST_NOTIFICATIONS
                ) != PackageManager.PERMISSION_GRANTED
                ) {
                    // Si no hay permiso, lo pedimos
                    ActivityCompat.requestPermissions(
                        this@MainActivity,
                        permissions = arrayOf(Manifest.permission.POST_NOTIFICATIONS),
                        requestCode = 101
                    )
                    return@withContext
                }

                mostrarNotificacion( titulo = "Reseña editada", mensaje = "Has editado: $titulo")
            }
        }
    }
}

```

```

    }

    }

    dialog.dismiss()

    }

    }

    }

    botonBorrar.setOnClickListener {
        lifecycleScope.launch( context = Dispatchers.IO) {
            reviewDao.delete( review = resena)
            filtrarLista(generoFiltro.selectedItem.toString())
        }
        dialog.dismiss()
    }

    }

    dialog.show()

}

```

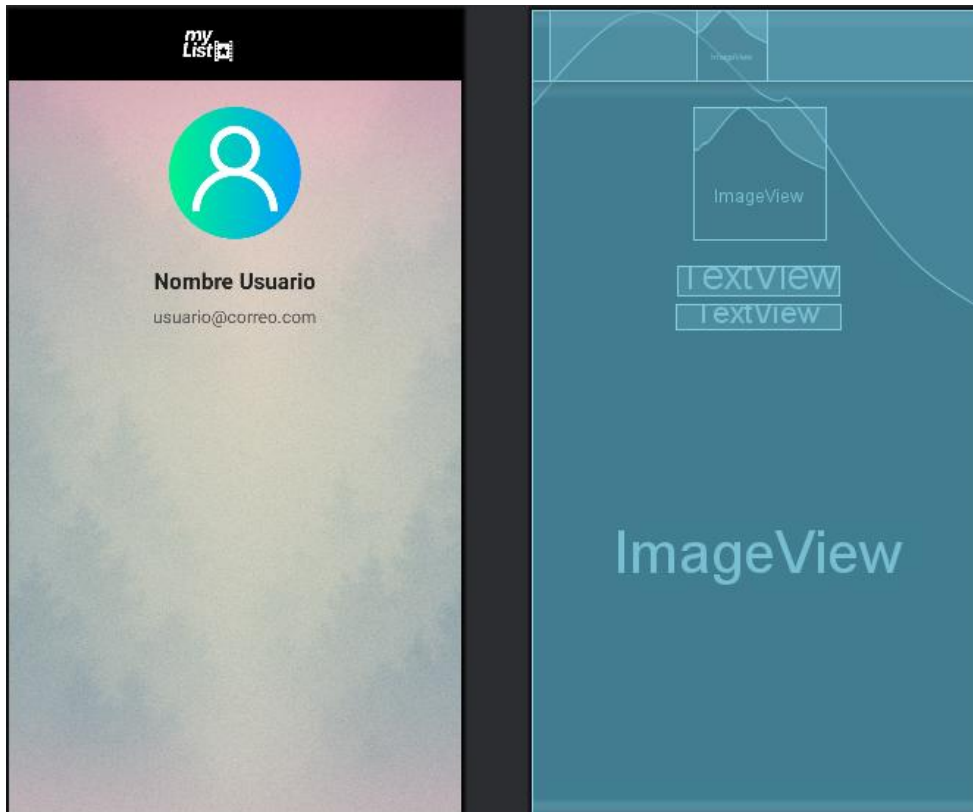
- Carga los datos existentes en un diálogo editable, permite guardar los cambios, eliminar la reseña o cancelar la operación, y estas modificaciones se actualizan en la base de datos y se refresca el listado, tiene una pequeña validación en los campos, no pueden estar vacíos, y una vez editada la reseña, el hilo principal, verifica si la app tiene permiso para enviar notificaciones, si no lo tiene, lo solicita al usuario, y si sí lo tiene, muestra una notificación con el título “Reseña editada” y un mensaje diciendo “Has editado: x película/serie”, y si se le hace click al boton de borrar, entonces se borrará de la base de datos mediante el método delete del reviewdao, y se actualice la lista de reseñas.

ProfileActivity:

Qué hace: Muestra la información del usuario, como su nombre y correo, y permite volver a la pantalla anterior (MainActivity).

Layout asociado (diseño): activity_profile.xml

```
setContentView(R.layout.activity_profile)
```



Librerías:

- **import android.graphics.Color:** Permite usar y manipular colores en la interfaz de Android.
- **import android.os.Bundle:** Proporciona una forma de pasar datos entre actividades o guardar el estado de la app.
- **import androidx.appcompat.app.AppCompatActivity:** Permite crear actividades que sean compatibles con versiones antiguas de Android.
- **import androidx.appcompat.widget.Toolbar:** Permite usar una barra de herramientas personalizable en la interfaz.
- **import android.widget.TextView:** Permite mostrar texto en la pantalla de la app.
- **import com.google.firebase.auth.FirebaseAuth:** Proporciona funcionalidades para autenticación de usuarios con Firebase.

Componentes:

- FirebaseAuth auth: Instancia para manejar la autenticación de usuarios.

```
auth = FirebaseAuth.getInstance()
```

- Toolbar toolbar: Barra superior con botón de volver.

```
val toolbar = findViewById<Toolbar>( id = R.id.toolbarProfile)
setSupportActionBar(toolbar)
supportActionBar?.setDisplayHomeAsUpEnabled(true)
supportActionBar?.setDisplayShowTitleEnabled(false)
toolbar.navigationIcon?.setTint(Color.WHITE)
toolbar.setNavigationOnClickListener { finish() }
```

- TextView textName: Muestra el nombre del usuario extraído del correo.

```
val textName = findViewById<TextView>( id = R.id.textName)
```

- TextView textEmail: Muestra el correo electrónico del usuario.

```
val textEmail = findViewById<TextView>( id = R.id.textEmail)
```

Flujo:

- Al iniciar la actividad (onCreate):
 - Se obtiene la instancia de FirebaseAuth y el usuario actual (currentUser).
 - Se configura la Toolbar con botón de volver y color del ícono.
 - Se obtienen los TextView de nombre y correo.
- Si el usuario está logueado:
 - Se muestra su correo en textEmail, y se extrae el nombre del usuario a partir del correo (todo lo que está antes de “@”) y se muestra en textName.

```

currentUser?.let {
    val email = it.email ?: "usuario@ejemplo.com"
    textEmail.text = email

    val nombre = email.substringBefore(delimiter = "@")
    textName.text = nombre
}

```

- Botón de volver de la Toolbar: cierra la actividad (finish()).

```

toolbar.setNavigationOnClickListener { finish() }

```

AndroidManifest.xml

Qué hace: Declara las actividades y define cuál actividad “clase” es la de inicio.

Permisos:

- La aplicación solicita permiso para enviar notificaciones al usuario.

```

<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

```

Actividades:

SplashActivity: Primera pantalla al abrir la app.

LoginActivity: Se abre desde SplashActivity o RegisterActivity.

RegisterActivity: Solo se abre desde LoginActivity.

MainActivity: Se abre desde Login o Register o desde el SplashActivity.

ProfileActivity: Se abre desde el MainActivity.

```
<!-- SplashActivity -->
<activity
    android:name=".SplashActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

<!-- LoginActivity -->
<activity
    android:name=".LoginActivity"
    android:exported="true" />

<!-- RegisterActivity -->
<activity
    android:name=".RegisterActivity"
    android:exported="false" />

<!-- MainActivity -->
<activity
    android:name=".MainActivity"
    android:exported="false" />

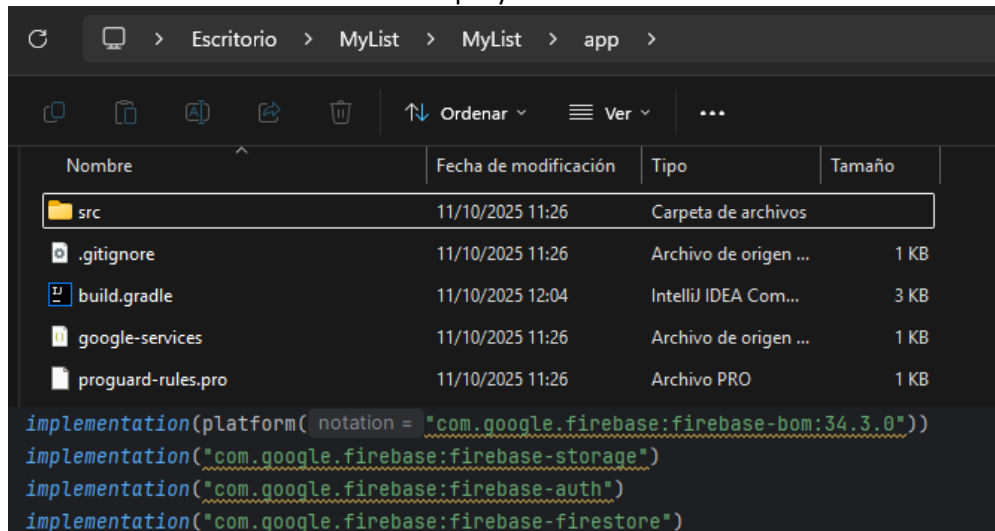
<!-- ProfileActivity -->
<activity
    android:name=".ProfileActivity"
    android:exported="false"/>
```

6. LAYOUTS (DISEÑO)

- **activity_login.xml** (LoginActivity): Es la interfaz de la actividad de inicio de sesión. Contiene una ImageView (logo), dos EditText (correo electrónico y contraseña), un TextView con el texto "¿Has olvidado tu contraseña?", y los botones Registrar e Iniciar sesión.
- **activity_register.xml** (RegisterActivity): Es la interfaz de la actividad de registro. Incluye una ImageView (logo), tres EditText (correo electrónico, contraseña y repetir contraseña), y los botones Registrar y Volver al login.
- **activity_main.xml** (MainActivity): Es la interfaz de la actividad principal de la aplicación. Presenta un fondo personalizado, una barra superior (TopBar), una ImageView (logo), el botón de cerrar sesión, un Spinner para filtrar por género, un RecyclerView para mostrar las reseñas, y el botón para agregar una nueva reseña.
- **activity_profile.xml** (ProfileActivity): Define el diseño de la pantalla de perfil del usuario, contiene una Toolbar con botón de volver y dos TextView: uno para mostrar el nombre del usuario y otro para mostrar su correo electrónico.
- **dialog_resena.xml** (MainActivity): Define un diálogo personalizado para guardar y editar las reseñas, muestra tres TextView: Título, Género y Comentario.
- **item_resena.xml** (ReviewAdapter): Define el diseño de cada reseña individual dentro del RecyclerView. Se muestra en una card con tres TextView: Título, Género y Comentario.

7. PROBLEMAS ENCONTRADOS Y SUS SOLUCIONES

- Nos encontramos con que el archivo google-services.json estaba mal colocado, debía de estar dentro de la carpeta app del proyecto, y al no estar allí, Firebase no se inicializaba y todas las llamadas a FirebaseAuth fallaban, después de esto detectamos dependencias de Firebase incompatibles, algunas versiones de firebase-auth, firebase-core y otras no coincidían, y estas generaban ciertos errores en compilación y en tiempo de ejecución. Lo solucionamos asegurándonos de usar en build.gradle las versiones compatibles entre sí y con la versión de Android de nuestro proyecto.



- Nos encontramos con un problema en la base de datos, al principio teníamos la versión 1, pero al realizar cambios en la estructura de UserReview (la tabla), la aplicación empezaba a dar errores en la base de datos debido a una incompatibilidad con la tabla existente, para solucionarlo lo que hicimos fue incrementar la versión en AppDatabase.

```
@Database(entities = [UserReview::class], version = 2, exportSchema = false)
```

8. DIAGRAMA DE CLASES Y ARQUITECTURA.

Diagrama de clases:

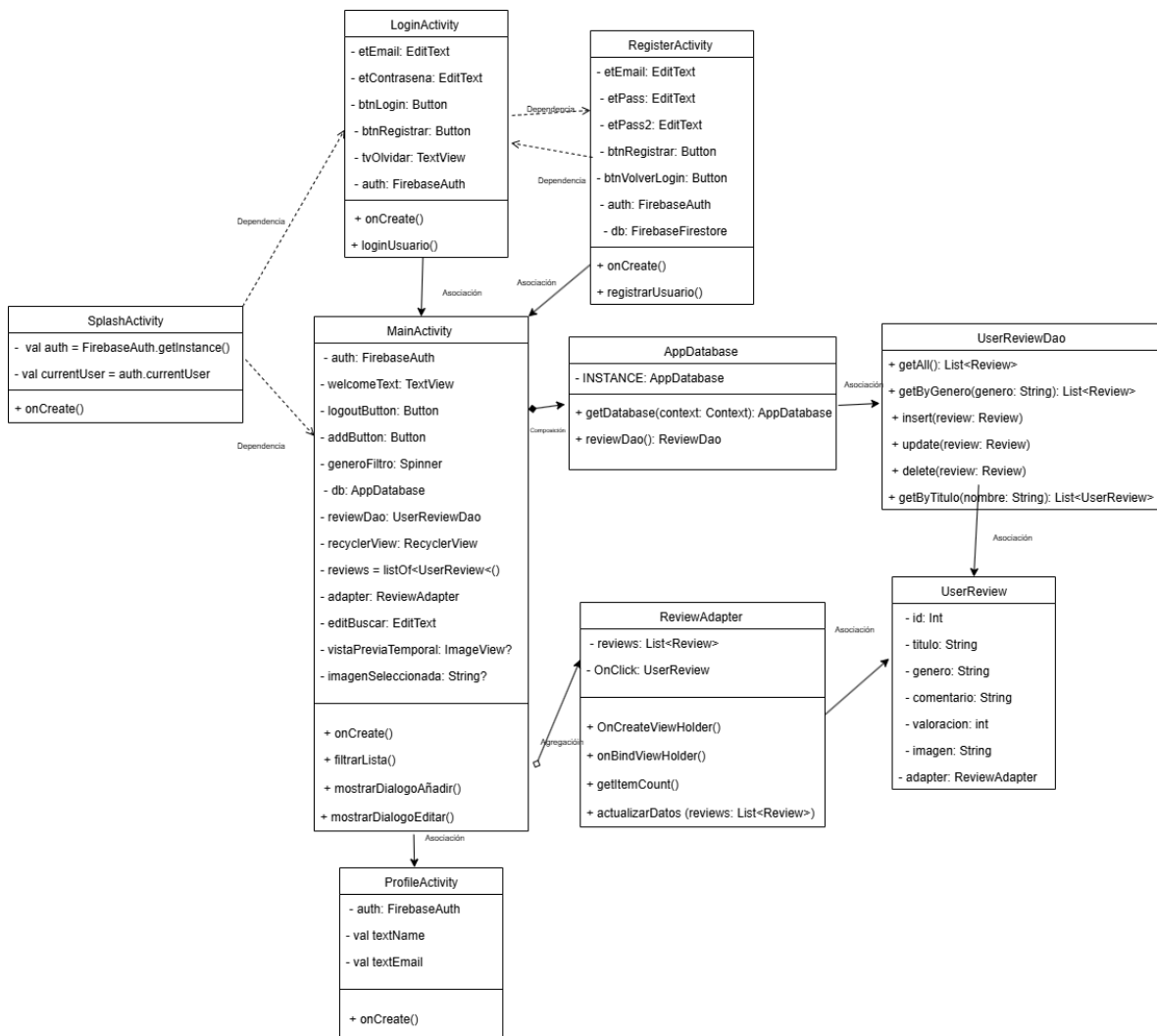
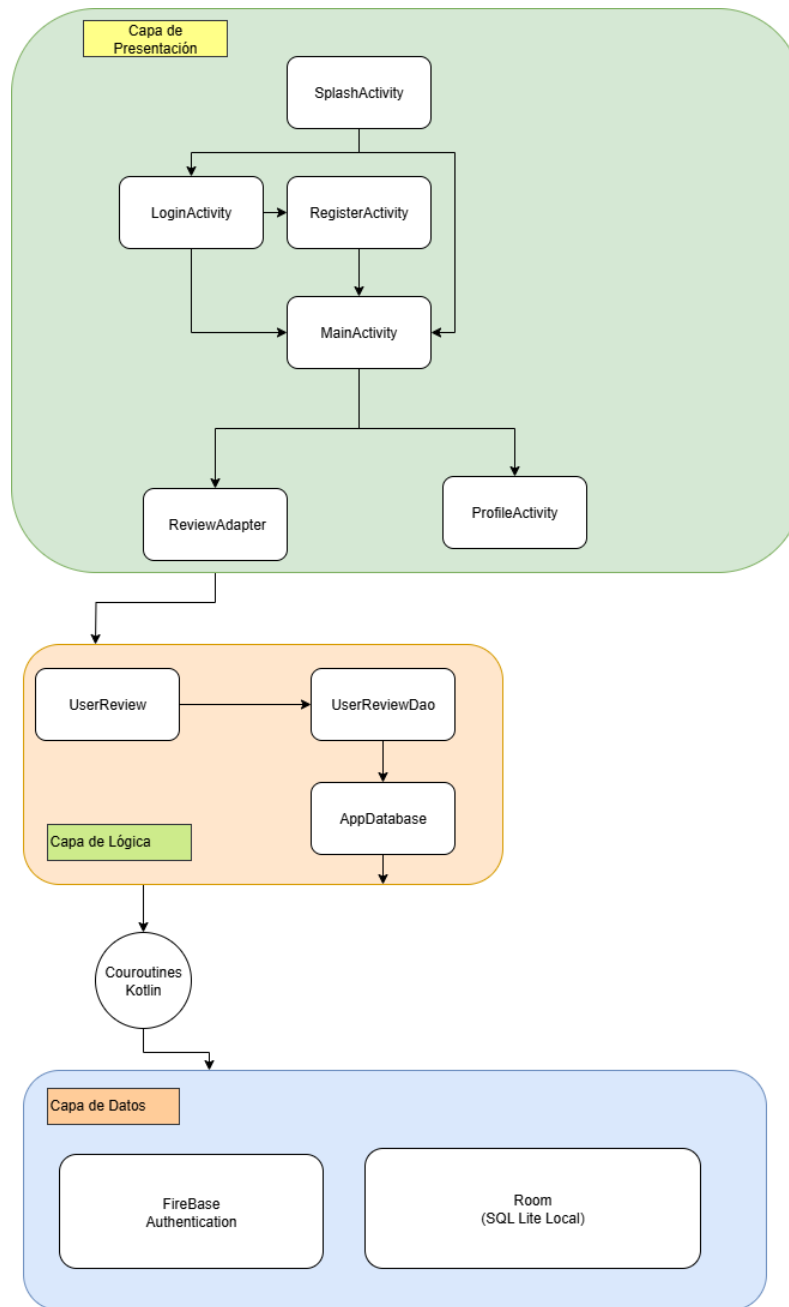


Diagrama de arquitectura:



10. REQUISITOS MINIMOS DE LA APP

Estos detalles se designan en el **build.gradle.kts** a nivel de **app**, excepto la ram y otros pequeños detalles que se pondrían como comentario en el AndroidManifest.xml con <meta-data, y luego ya se realizaría un detector de ram en Splash para hacer la comprobación de ram necesaria, y así ejecutar o no la app.

Android: 7.0 (API 24) o superior (**minSdk**)

Librerías: compatibles con API 24+

Memoria RAM: 1 GB

Pantalla: cualquier resolución soportada por Android 7.0 en adelante.

```
defaultConfig {  
    applicationId = "com.example.mylist"  
    minSdk = 24  
    targetSdk = 36  
    versionCode = 1  
    versionName = "1.0"  
  
    testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"  
}
```